

Storage Wars

Write it down first: the log that makes COMMIT mean something

Issue #4

Sun, 04 Oct 2026

~19 min read

Level 3/5

⚡ The short version

1. **A commit is durable when its log record is on disk, not its data pages.** Part 1 follows a write through the write-ahead log, a checkpoint and a crash, and shows which settings trade safety for speed.
2. **Patch your pooler:** PgBouncer 1.26.0 fixes **three denial-of-service CVEs**, two of them triggerable by **unauthenticated clients**.
3. **The log itself is getting more attention:** MariaDB 13.0 can now **archive the full InnoDB redo log** for point-in-time recovery, and Valkey 9.2.0-rc1 adds **forkless snapshots**.

Release radar

18.6

current **PostgreSQL** minor; **19** is still at **Beta 4**, with an RC "expected early October"

4.3.1

latest **Apache Kafka** feature release; **4.4.0** is not out yet

39 days

until **PostgreSQL 14** end-of-life on 12 Nov 2026

Sources: postgresql.org, PostgreSQL 19 Beta 4 announcement, Kafka downloads, PostgreSQL versioning policy.

PART 1 Learn



The write-ahead log: durability, crash recovery and fsync

FOUNDATIONS

~10 MIN

LEVEL 3

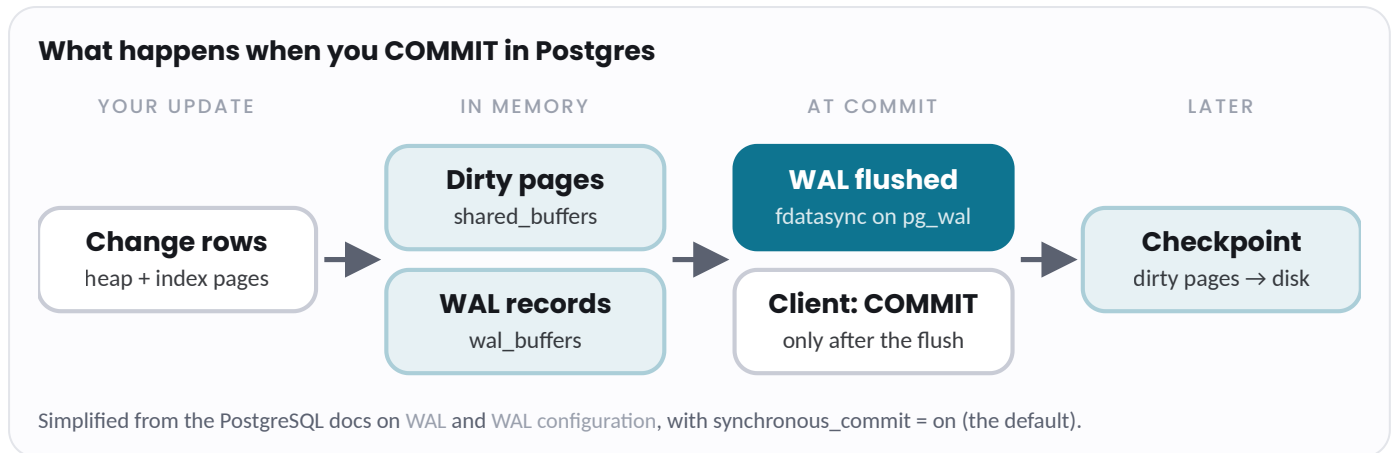
When Postgres says `COMMIT`, your changed rows usually haven't reached their data files yet. They're sitting in the buffer pool from Issue #1, in RAM. If the power goes out a millisecond later, the commit still survives. The reason is one rule that nearly every serious database follows: **write down what you're about to do, durably, before you do it.**

THE RULE

The Postgres docs put it in one sentence: changes to data files must be written **only after** the WAL records describing them **have been flushed to permanent storage**. After a crash, anything missing from the data files is **redone from the WAL**. That's roll-forward recovery, or **REDO**. (PostgreSQL docs)

Why is that faster than just writing the pages? Because a transaction might touch ten pages scattered across a table and three indexes. **Flushing them all at commit means many random writes. The WAL is one file written sequentially**, so only it has to be flushed, and the docs note that one `fsync` of the WAL can commit

many small concurrent transactions at once. (PostgreSQL docs) The data pages are written later, lazily, in big batches.



EVERY CHANGE GETS AN ADDRESS: THE LSN

The WAL is a single, ever-growing stream of records. A record's position is its **log sequence number (LSN)**: a **byte offset into the WAL that only ever increases**. On disk the stream is cut into **16 MB segment files** in `pg_wal`, named from `000000010000000000000001` upwards, and the names **never wrap**. (PostgreSQL docs: WAL internals)

The LSN is how everything else in Postgres talks about "how far along" it is. Postgres tracks separate **insert, write and flush** positions, and only the flush position counts as **"written to durable storage"**. Replicas report how far they've received, flushed and replayed. And `pg_wal_lsn_diff()` turns two LSNs into a byte count, which is how most replication-lag dashboards are built. (PostgreSQL docs: admin functions)

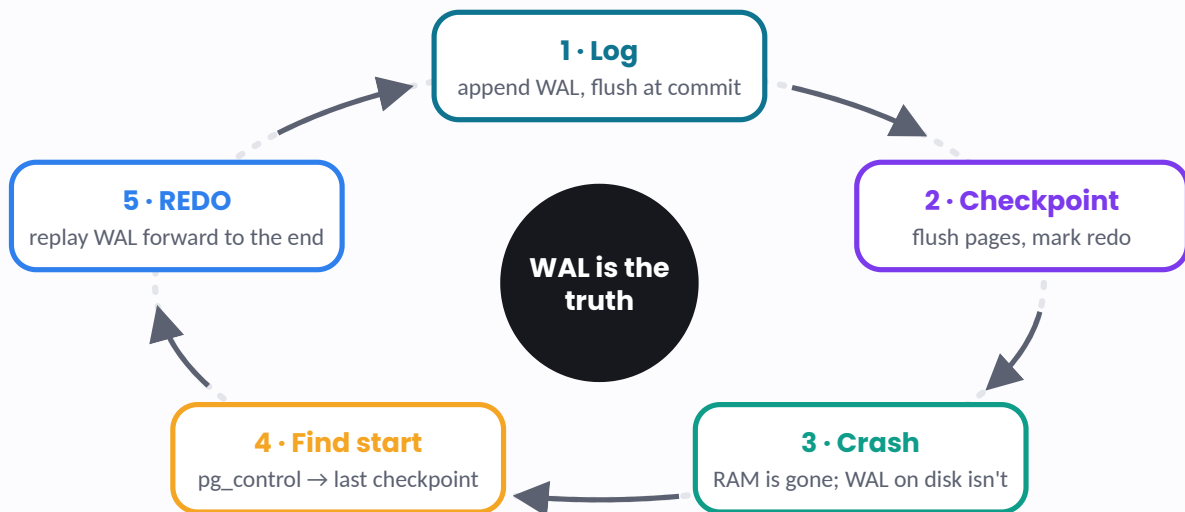
CHECKPOINTS: HOW THE LOG STAYS SHORT

If dirty pages were never written back, the WAL would grow forever and recovery would replay from the beginning of time. A **checkpoint** fixes that. It flushes **all dirty data pages** to disk and writes a **checkpoint record** to the WAL. **Everything before that record's redo point is guaranteed to be on disk**, so older segments can be recycled. (PostgreSQL docs)

A checkpoint starts every `checkpoint_timeout` (**default 5 min**) or when the WAL is about to exceed `max_wal_size` (**default 1 GB**), whichever comes first. With `checkpoint_completion_target = 0.9` (the default), Postgres spreads the page writes over about 90% of the interval to avoid an I/O spike. (PostgreSQL docs)

The trade-off is crash-recovery time versus steady-state cost. Longer gaps between checkpoints mean fewer page writes but more WAL to replay after a crash. Shorter gaps have a hidden cost too, which brings us to torn pages.

Normal running, crash, recovery



Recovery reads `pg_control`, then the checkpoint record, then replays forward from its redo location (PostgreSQL docs).

THE TORN PAGE PROBLEM

A Postgres data page is **8 kB**, but disks write in smaller sectors. If the power fails mid-write, you can end up with a **page that's half old and half new**. A WAL record that says "change row 7" can't repair that, because the page it would be applied to is garbage.

Postgres's fix is `full_page_writes` (on by default): the **first time a page is modified after a checkpoint, the whole page goes into the WAL**. Recovery can restore the full image and replay later changes on top. (PostgreSQL docs: WAL settings) InnoDB solves the same problem differently, with a **doublewrite buffer**. (MySQL docs)

Two answers to torn pages

🐘 Postgres: full-page images

- First change to a page after a checkpoint logs the whole 8 kB page
- Recovery restores the image, then replays later records
- Cost: WAL volume jumps right after each checkpoint

```
full_page_writes = on
```

VS

🐋 InnoDB: doublewrite buffer

- Pages go to a doublewrite area first, then to their real place
- A torn page is repaired from its good copy on recovery
- Written as one big sequential chunk with a single `fsync()`

```
innodb_doublewrite = ON
```

From the PostgreSQL WAL settings and InnoDB doublewrite buffer docs. Both can be turned off on storage that guarantees atomic page writes (the Postgres docs mention ZFS).

THE KNOB EVERYONE TOUCHES: WHEN DOES COMMIT RETURN?

Every engine lets you trade a little durability for speed. **The important question is what you lose: recent commits, or the whole database.**

Engine	Setting (default)	Relaxed option	What a crash can cost
PostgreSQL	<code>synchronous_commit = on</code>	<code>off</code>	Up to $3 \times$ <code>wal_writer_delay</code> (200 ms default) of commits; no corruption
PostgreSQL	<code>fsync = on</code>	<code>off</code>	Unrecoverable corruption. Only for data you can recreate
MySQL InnoDB	<code>innodb_flush_log_at_trx_commit = 1</code>	<code>2</code> or <code>0</code>	About 1 second of transactions
Redis AOF	<code>appendfsync everysec</code>	<code>no</code> (always is safer)	About 1 second of writes
Kafka	flushing left to the OS	n/a	Covered by replication to the in-sync replicas

Sources: [PostgreSQL async commit](#), [PostgreSQL WAL settings](#), [InnoDB parameters](#), [Redis persistence](#), [Kafka design](#). Redis AOF itself is off unless you set `appendonly yes`.

The Postgres docs are precise about the difference. With `synchronous_commit = off`, a crash can lose some recent commits, but **the database ends up as if those transactions had been aborted cleanly**. The risk is **data loss, not data corruption**. With `fsync = off`, all bets are off. ([PostgreSQL docs](#)) And `synchronous_commit` can be set **per transaction**, so you can relax it only for the writes you can afford to lose, like click logs, and keep payments fully synchronous:

```
BEGIN;
SET LOCAL synchronous_commit = off; -- this transaction only
INSERT INTO page_views (user_id, url) VALUES (42, '/pricing');
COMMIT; -- returns before the WAL flush
```

Kafka is the odd one out by design. It writes every message to the log immediately "**without necessarily flushing to disk**", and treats a message as committed only once **every in-sync replica has it in its log**. ([Kafka design](#)) **Durability there comes from copies on several machines, not from `fsync`**. Kafka II will cover what `acks` and `min.insync.replicas` actually promise.

In the real world: GitLab, 31 January 2017

GitLab's secondary database fell behind, and **the primary removed WAL segments before the secondary could replicate them**. GitLab **wasn't using WAL archiving**, so the replica couldn't catch up and had to be rebuilt. During that work, an engineer ran a delete of the data directory on the **primary by mistake**; about **300 GB** was gone before it was stopped. The backups turned out not to work: `pg_dump` had been failing silently because it was a **9.2 client against a 9.6 server**. GitLab restored from an **LVM snapshot taken 6 hours earlier**. About **5,000 projects, 5,000 comments and 700 new user accounts** were lost, and the site was down for around 18 hours. ([GitLab postmortem](#)) The lesson for today: **the WAL is your recovery story, so archive it and test restoring from it.**

💣 Gotcha: when fsync() lies

In 2018 Postgres developers found that on Linux, a failed `fsync()` could mark the dirty pages clean anyway. Before kernel 4.13, retrying `fsync()` could report success even though the data was never written. Postgres used to retry. Since then it **PANICs on `fsync()` failure** and recovers from the WAL instead, a fix backpatched as far as 9.4. InnoDB and WiredTiger made similar changes. (PostgreSQL wiki: "fsyncgate 2018") A 2020 USENIX study found that ext4, XFS and Btrfs **all mark pages clean after a failed `fsync`**, and that none of the applications it tested, including PostgreSQL, SQLite and Redis, handled every failure correctly. (USENIX ATC '20)

The quieter version of the same problem is a **drive or RAID cache that acknowledges a write before it's durable**. The docs warn that volatile write caches **lose their contents in a power failure**, so check that your storage either has a battery or capacitor-backed cache or has the write cache turned off. (PostgreSQL docs: reliability)

WHY IT MATTERS TO YOU



Day-to-day impact



Interview value



Production risk



USE IT TOMORROW

1. **Never set `fsync = off`** on data you can't recreate. Use `synchronous_commit = off`, per transaction if possible, when you want speed.
2. **Archive your WAL** (or use a tool that does, like pgBackRest or your cloud's PITR) and **test a restore**.
3. **Watch checkpoint frequency**. If checkpoints are triggered by `max_wal_size` rather than the timer, raise it.
4. **Track replication lag in bytes** with `pg_wal_lsn_diff()` and alert before the primary recycles WAL a replica still needs.
5. **Know your storage's write cache**. A lying cache defeats every setting in the table above.



Under the hood

- **Every WAL record carries a CRC-32C checksum**, and full-page images are always checksum-protected, so recovery stops at the first torn or garbage record rather than replaying it. (PostgreSQL docs: reliability)
- **`wal_sync_method` defaults to `fdasync` on Linux**. `pg_test_fsync` compares the methods on your hardware, though the docs warn the difference often doesn't matter in practice. (PostgreSQL docs), (`pg_test_fsync`)
- **Group commit is tunable**. `commit_delay` (default 0) makes a committing backend wait briefly so others can share its flush, but only when at least `commit_siblings` (default 5) transactions are active. The docs suggest starting at **half the single-flush time** that `pg_test_fsync` reports. (PostgreSQL docs)
- **`wal_buffers` defaults to 1/32 of `shared_buffers`**, at least 64 kB and at most one segment (16 MB). `wal_compression` (off by default) can shrink full-page images with `pglz`, `lz4` or `zstd`. (PostgreSQL docs)
- **Physical logs replicate physical changes**. Uber's 2016 write-up described how one updated column in a table with many indexes produced WAL for every index, and how that WAL volume strained cross-data-centre links. (Uber Engineering)

Try it (15 minutes, Docker)

```
docker run -d --name pg -e POSTGRES_PASSWORD=pw postgres:18
docker exec -it pg psql -U postgres
```

```
CREATE TABLE t (id int, v text);
SELECT pg_current_wal_lsn() AS before \gset
INSERT INTO t SELECT g, md5(g::text)
  FROM generate_series(1, 100000) g;
-- How much WAL did 100k rows cost?
SELECT pg_size_pretty(
  pg_wal_lsn_diff(pg_current_wal_lsn(), :'before'));
SELECT pg_walfile_name(pg_current_wal_lsn()); -- copy this
\q
```

```
# 1. What's inside? Record types and full-page images
docker exec pg sh -c 'cd "$PGDATA/pg_wal" && pg_waldump --stats <file>'

# 2. Commit speed, synchronous_commit on vs off
docker exec pg pgbench -U postgres -i postgres
docker exec pg pgbench -U postgres -c 8 -T 20 postgres
docker exec -e PGOPTIONS='-c synchronous_commit=off' pg \
  pgbench -U postgres -c 8 -T 20 postgres

# 3. Pull the plug, then watch crash recovery replay the WAL
docker kill pg && docker start pg && docker logs pg | tail -8
```

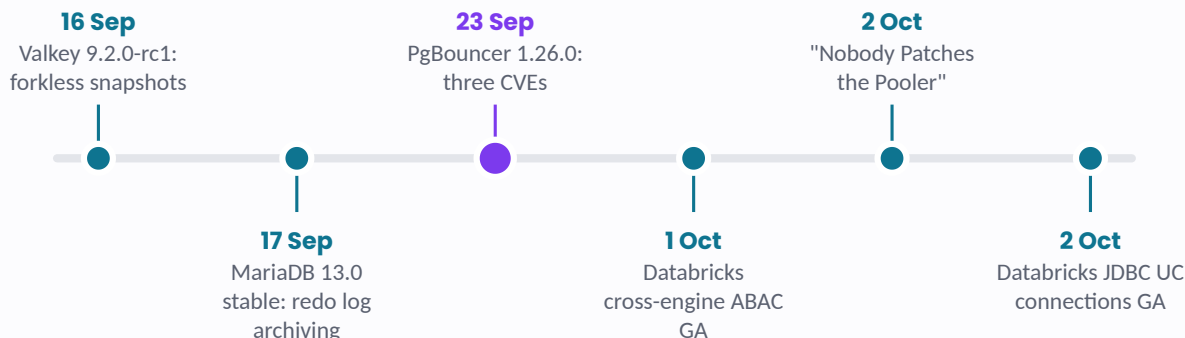
Compare the transactions per second in step 2. The gap is what each WAL flush costs on your disk. In step 3, look for the log lines about **automatic recovery** and **redo**. An error at the end of `pg_waldump` on the current file is normal: it's still being written.

GO DEEPER

- **Reliability and the Write-Ahead Log** (PostgreSQL docs). The whole chapter, from disk caches to async commit and checkpoints, is short and excellent.
- **Fsync Errors** (PostgreSQL wiki). The fsyncgate story, kernel by kernel.
- **ARIES** (Mohan et al., 1992). The paper behind most modern WAL-based recovery: LSNs on pages, redo, then undo.
- **Can Applications Recover from fsync Failures?** (USENIX ATC '20). What really happens below your database when a write fails.

~9 min · A pooler you probably forgot to patch, the redo log grows up, and Valkey learns to snapshot without forking.

The last three weeks in data



Top story 1 · PgBouncer 1.26.0 fixes three denial-of-service CVEs

SECURITY

POSTGRES

What happened. PgBouncer **1.26.0** shipped on **23 September** with fixes for three CVEs, all denial of service.

(PostgreSQL.org)

- **CVE-2026-19888**: a crash, triggerable by **unauthenticated clients**, via a SCRAM client-final-message without a nonce.
- **CVE-2026-6668**: an infinite loop, triggerable by **unauthenticated clients**, via an integer overflow in packet-buffer growth.
- **CVE-2026-6669**: unbounded work during login, triggered by a **malicious PostgreSQL server** sending a huge SCRAM iteration count.

The release also adds `pool_idle_timeout`, lets `query_wait_timeout` be set per user and database, tracks `search_path` and `default_transaction_read_only` by default, and **removes online restart (-R)**. (PostgreSQL.org)

On 2 October, Christophe Pettus pointed out that the SCRAM crash has existed **since SCRAM support arrived in 1.11.0 in 2019**, and that poolers tend to be the component nobody remembers to upgrade. (The Build)

Why it matters. A pooler sits in front of **every** connection. If it falls over, the database might as well be down. Two of the three bugs need **no credentials at all**, so anything that can reach the PgBouncer port can try them.

👁️ **What to do.** Run `SHOW VERSION;` on the PgBouncer admin console of every pool, including sidecars baked into container images. Upgrade to 1.26.0. If you use `-R` for online restarts, plan a replacement (for example, running instances side by side) before you upgrade.

WHY IT MATTERS TO YOU



Day-to-day impact



Act-now urgency



Production risk



Top story 2 · MariaDB 13.0 keeps the whole redo log

RELEASE

WAL

What happened. MariaDB 13.0.2 became the first stable 13.0 release on **17 September**. It's a **rolling release**, **not an LTS**. The headline features are **InnoDB log archiving** and a **DuckDB storage engine** (gamma, not yet recommended for production) that lets you join operational and analytical tables in one server. (MariaDB Foundation)

Log archiving is today's Learn topic in action. Normally the InnoDB redo log is a **ring buffer**, overwritten once a checkpoint makes old records unnecessary. With `innodb_log_archive=ON`, InnoDB writes a continuous sequence of files named `ib_<lsn>.log` instead, to "**retain the full InnoDB write-ahead log history**" for **point-in-time recovery and incremental backups**. It's **off by default**. (MariaDB docs)

The catch: the docs say **no shipped backup tool can create or restore backups in this format yet**, and `mariadb-backup` **fails** when the server runs with archiving on. (MariaDB docs)

Why it matters. It moves InnoDB closer to how Postgres has long done PITR: keep every log record, restore a base backup, replay to a point in time. For now it's groundwork, not something to switch on in production.

WHY IT MATTERS TO YOU



Day-to-day impact



Act-now urgency



Production risk



Top story 3 · Valkey 9.2 release candidate: snapshots without fork()

RC

VALKEY

What happened. Valkey 9.2.0-rc1 (16 September) adds **forkless RDB snapshots**, opt-in via the new `forkless-infrastructure-enabled` and `bgsave-default-method` configs. It also adds whole-file **LZ4 compression for RDB files** (`rdbcompression lz4`), optional **LZ4 compression of the replication stream** (`repl-compression`), and `replication-throttling-enabled`, which slows client writes when replicas fall behind so they don't disconnect and need a full resync. (Valkey release notes)

Two behaviour notes: **active expiry now counts as a change**, so save points and `BGSAVE` **may trigger more often**, and a race that **could produce corrupt, unloadable RDB files** when compression ran concurrently is fixed. (Valkey release notes)

Why it matters. Yesterday's issue showed that `fork()` **pauses grow with memory size**, up to seconds on large instances. Avoiding the fork for snapshots goes straight at one of the biggest latency spikes in Redis-style servers. It's an RC, so test it, don't ship it, and wait for GA benchmarks.

WHY IT MATTERS TO YOU



Day-to-day impact



Act-now urgency



⚡ Quick hits

🔒 Databricks: cross-engine ABAC is GA (1 Oct).

External engines can now read Unity Catalog managed tables with ABAC policies, row filters and column masks enforced. **JDBC Unity Catalog connections** went GA on 2 Oct, with SSL certificate support. (Databricks release notes)

🔗 Aurora DSQL gets foreign keys.

All the usual actions (`CASCADE` , `SET NULL` ...), checked against the transaction snapshot **at commit, without locks**; violations come back as serialization errors. AWS warns of extra reads on DML. (InfoQ), (AWS)

🌊 Snowflake Openflow gen 1 → gen 2 migration is GA (1 Oct).

A deployment and all its runtimes migrate together. (Snowflake docs)

📅 Coming up.

PostgreSQL 19 RC ("expected early October", with GA possibly this month), **Kafka 4.4.0** (planned no earlier than 9 Sep; [release plan](#)), **DuckDB 2.0** in October, and the **12 Nov** Postgres minor releases, the last for PG 14.

🐘 ClickHouse WalShadow (10 Sep).

Replicates Postgres into ClickHouse by decoding the **physical WAL** outside the source, using a schema-only shadow Postgres and parallel Rust decoders. ClickHouse claims about **200 ms** commit-to-visible versus about 10 s for PeerDB. Open source. (ClickHouse)

📊 Uber splits M3DB into subclusters.

In the old placement one node could share data with up to **66.67%** of the cluster; fixed-size subclusters now own non-overlapping shards, limiting the blast radius. (InfoQ)

🐪 MongoDB: substring search on encrypted fields is GA.

Announced at .local NYC (30 Sep): Queryable Encryption now supports **prefix, suffix and substring** queries, alongside Atlas Ephemeral Clusters in preview. (MongoDB)

📌 Revision card

- **Log first, data later:** a commit is durable once its **WAL record is flushed**; data pages are written later.
- The WAL is **sequential**, so one flush can commit **many transactions** (group commit).
- The **LSN** is a byte offset into the WAL; `pg_wal_lsn_diff()` turns two of them into lag in bytes.
- **Checkpoints** (every 5 min or 1 GB by default) bound recovery time; recovery **replays forward from the last checkpoint's redo point**.
- **Torn pages:** Postgres logs **full-page images** after each checkpoint; InnoDB uses a **doublewrite buffer**.
- `synchronous_commit = off` can lose recent commits; `fsync = off` can corrupt everything.
- A failed `fsync()` may already have **dropped your data**, so Postgres now **PANICs** and recovers from the WAL.
- **Upgrade PgBouncer to 1.26.0:** two of its three CVEs need no authentication.

Storage Wars is researched and written by Claude every morning from official docs and release notes, project blogs and engineering write-ups. Vendor numbers are labelled as claims, and every item links to its source. Want more Kafka, less Mongo, or a different depth? Change `data/settings.yml` or just reply. 📧